

MFML ASSIGNMENT 1

Student ID: 2026AK05157

Submitted by: KV GAUTHAMI

```
import random
import numpy as np
random.seed(246) # Setting a random seed for reproducibility.

# =====
# Helper Functions
# =====

def get_random_float():
    # Generating randoms in the format n.dddddddd (8 digits after
    decimal)
    return round(random.uniform(1.0, 9.99999999), 8)

def print_matrix(mat, name="Matrix"):
    print(f"\n--- {name} ---")
    if mat is None:
        print("None")
        return
    for row in mat:
        print(["{:.8f}".format(val) for val in row])
```

Q1. Finding solutions of linear systems

1.1 Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing REF, and RREF without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b. Note that this part should be explicitly there in the code and give a comment line on the same.

```
import random
import numpy as np
random.seed(246) # Setting a random seed for reproducibility.

# =====
# Helper Functions
# =====

def get_random_float():
```

```

    # Generating randoms in the format n.dddddddd (8 digits after
    decimal)
    return round(random.uniform(1.0, 9.99999999), 8)

def print_matrix(mat, name="Matrix"):
    print(f"\n--- {name} ---")
    if mat is None:
        print("None")
        return
    for row in mat:
        print(["{:.8f}".format(val) for val in row])

# =====
# 1.1 Construct Matrix A[m x n] (m<n) and Vector b[m x 1]
# =====

# Hardcoding dimensions directly
m = 4
n = 5

print("\n\n")
if m <= 0 or n <= 0:
    print("Error: Cannot construct matrix. Dimensions 'm' and 'n' must
    be strictly greater than 0.")
elif m >= n:
    print("Error: Cannot construct matrix. The number of rows (m) must
    be strictly less than the number of columns (n).")
else:
    print(f"Using dimensions: {m}x{n}. Generating matrices...")

    # Constructing Matrix A (m x n) and Vector b (m x 1)
    A = [[get_random_float() for _ in range(n)] for _ in range(m)]

    # Constructing Vector b (m x 1) with random floats as
    # b is of dimension m x 1
    b = [[get_random_float()] for _ in range(m)]

    # =====
    # Constructing the augmented matrix [A|b]
    # =====

    def construct_augmented_matrix(A, b):
        aug_matrix = []
        for i in range(len(A)):
            row = A[i][:]
            row.append(b[i][0])
            aug_matrix.append(row)

```

```

    return aug_matrix

# =====
# 2. Row Echelon Form (REF)
# =====

def compute_REF(aug_mat, tol=1e-8):
    mat = [row[:] for row in aug_mat]
    rows, cols = len(mat), len(mat[0])
    lead = 0

    for r in range(rows):
        if lead >= cols:
            return mat

        # Proper fix for zero pivot - Row interchange.
        # Finding a row with a non-zero entry in the current
        # column to swap with.
        i = r
        while abs(mat[i][lead]) < tol:
            i += 1
            if i == rows:
                # If the whole column below is zero, there's
                # nothing to swap with.
                # Skip that column, it becomes a free column.
                i = r
                lead += 1
                if lead == cols:
                    return mat

        # Perform the row swap only if the pivot is on a different
        # row
        if i != r:
            mat[i], mat[r] = mat[r], mat[i]

        # Normalize the pivot row
        pivot_val = mat[r][lead]
        for j in range(cols):
            mat[r][j] /= pivot_val

        # Eliminate rows below the pivot
        for i in range(r + 1, rows):
            factor = mat[i][lead]
            for j in range(cols):
                mat[i][j] -= factor * mat[r][j]

        lead += 1

    return mat

```

```

# =====
# 3. Reduced Row Echelon Form (RREF)
# =====

def compute_RREF(ref_mat):
    if ref_mat is None:
        return None

    mat = [row[:] for row in ref_mat]
    rows = len(mat)
    cols = len(mat[0])

    for r in range(rows - 1, -1, -1):
        lead_col = -1
        for j in range(cols - 1):
            if abs(mat[r][j] - 1.0) < 1e-8:
                lead_col = j
                break

        if lead_col != -1:
            for i in range(r - 1, -1, -1):
                factor = mat[i][lead_col]
                for j in range(cols):
                    mat[i][j] = mat[i][j] - (factor * mat[r][j])

    return mat

# =====
# Outputs
# =====

aug_matrix = construct_augmented_matrix(A, b)

print_matrix(A, "Matrix A")
print_matrix(b, "Vector b")
print_matrix(aug_matrix, "Augmented Matrix [A|b]")

ref_matrix = compute_REF(aug_matrix)
print_matrix(ref_matrix, "Row Echelon Form (REF)")

rref_matrix = compute_RREF(ref_matrix)
print_matrix(rref_matrix, "Reduced Row Echelon Form (RREF)")

print("\n\n")

```

Using dimensions: 4x5. Generating matrices...

```
--- Matrix A ---
```

```

['9.34367297', '5.65009134', '7.48549565', '1.59101978', '4.96588180']
['6.18018152', '4.00588679', '6.03414293', '7.16624093', '8.50769760']
['1.87849285', '7.19616802', '4.09569366', '2.94095457', '9.95570767']
['7.10553352', '5.04140597', '1.84313517', '9.72386355', '1.26426491']

--- Vector b ---
['2.87422558']
['1.89955866']
['8.13257100']
['8.82096748']

--- Augmented Matrix [A|b] ---
['9.34367297', '5.65009134', '7.48549565', '1.59101978', '4.96588180',
'2.87422558']
['6.18018152', '4.00588679', '6.03414293', '7.16624093', '8.50769760',
'1.89955866']
['1.87849285', '7.19616802', '4.09569366', '2.94095457', '9.95570767',
'8.13257100']
['7.10553352', '5.04140597', '1.84313517', '9.72386355', '1.26426491',
'8.82096748']

--- Row Echelon Form (REF) ---
['1.00000000', '0.60469703', '0.80112988', '0.17027777', '0.53146999',
'0.30761196']
['0.00000000', '1.00000000', '4.02983234', '22.74942589',
'19.43489940', '-0.00572692']
['-0.00000000', '-0.00000000', '1.00000000', '6.19513680',
'4.98478916', '-0.34764450']
['0.00000000', '0.00000000', '0.00000000', '1.00000000', '0.50460387',
'0.12519354']

--- Reduced Row Echelon Form (RREF) ---
['1.00000000', '0.00000000', '0.00000000', '0.00000000', '-
1.32482040', '0.17470958']
['0.00000000', '1.00000000', '0.00000000', '0.00000000', '0.46520508',
'1.67264309']
['-0.00000000', '-0.00000000', '1.00000000', '0.00000000',
'1.85869915', '-1.12323560']
['0.00000000', '0.00000000', '0.00000000', '1.00000000', '0.50460387',
'0.12519354']

```

1.2 Write a Python code to identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax=0$.

```

# =====
# 4. Finding the Pivots
# =====

```

```

def analyze_and_solve(rref_mat):
    # If our matrix failed in step 1 (division by zero), we can't
    solve it.
    if rref_mat is None:
        print("Cannot find solutions without a valid RREF matrix.")
        return None, None, None, None

    rows = len(rref_mat)
    # The actual number of columns 'n' is the total columns minus 1
    # (since the last column is our augmented vector 'b')
    cols = len(rref_mat[0]) - 1

    pivot_cols = []
    non_pivot_cols = []

    # -----
    # Step A: Identify Pivot and Non-Pivot columns
    # -----
    for r in range(rows):
        for c in range(cols):
            # We look for the leading 1 in each row.
            # Using a small tolerance (1e-8) for floating point
            inaccuracies.
            if abs(rref_mat[r][c] - 1.0) < 1e-8:
                pivot_cols.append(c)
                break # Found the pivot for this row, skip to the next
row

        # Any column index that isn't a pivot column is a non-pivot (free
        variable) column
        for c in range(cols):
            if c not in pivot_cols:
                non_pivot_cols.append(c)

    # -----
    # Step B: Particular Solution to x_p
    # -----
    # Assumption: To find a single valid particular solution, the
    simplest approach
    # is to set all free variables (non-pivot columns) to 0.
    particular_solution = [0.0] * cols

    for r in range(rows):
        p_col = -1
        # Locate the pivot column for the current row again
        for c in range(cols):
            if abs(rref_mat[r][c] - 1.0) < 1e-8:
                p_col = c

```

```

        break

    if p_col != -1:
        # If a pivot exists in this row, its value simply equals
the augmented
        # column's value (the very last item in the row) since
free variables are 0.
        particular_solution[p_col] = rref_mat[r][-1]

    # -----
    # Step C: Solutions to  $Ax = 0$  (Null Space)
    # -----
    # To find the solutions to the homogeneous equation, we need to
generate a basis vector
    # for each free variable.
    null_space_solutions = []

    for free_col in non_pivot_cols:
        # Initialize a new solution vector with zeros
        basis_vector = [0.0] * cols

        # We set exactly one free variable to 1, and leave the other
free variables as 0
        basis_vector[free_col] = 1.0

        # Now, we solve for the pivot variables in terms of this free
variable
        for r in range(rows):
            p_col = -1
            for c in range(cols):
                if abs(rref_mat[r][c] - 1.0) < 1e-8:
                    p_col = c
                    break

            if p_col != -1:
                # The equation is:  $1 \cdot (x_{\text{pivot}}) + (rref\_val) \cdot (x_{\text{free}}) =$ 
0
                # Therefore:  $x_{\text{pivot}} = -rref\_val \cdot x_{\text{free}}$ 
                # Since  $x_{\text{free}}$  is 1.0,  $x_{\text{pivot}}$  is simply the negative
of the RREF value.
                basis_vector[p_col] = -rref_mat[r][free_col]

        null_space_solutions.append(basis_vector)

    return pivot_cols, non_pivot_cols, particular_solution,
null_space_solutions

# =====
# Outputs for Part 2

```

```

# =====

pivots, non_pivots, part_sol, null_sols =
analyze_and_solve(rref_matrix)

num_free_vars = len(non_pivots)

if pivots is not None:
    print("\n\n")
    print(f"\nPivot Columns (0-indexed): {pivots}")
    print(f"Non-Pivot Columns (Free variables, 0-indexed):
{non_pivots}")
    print(f"Number of non-pivot columns: {len(non_pivots)}")
    print(f"Number of free variables present: {num_free_vars}")

    print("\n--- Particular Solution (x_p) ---")
    formatted_part_sol = ["{:.8f}".format(val) for val in part_sol]
    print(formatted_part_sol)

    print("\n--- Solutions to Ax = 0 (Null Space Basis Vectors) ---")
    if not null_sols:
        print("Only the trivial solution (all zeros) exists since
there are no free variables.")
    else:
        for i, sol in enumerate(null_sols):
            print(f"Vector v_{i+1}:", ["{:.8f}".format(val) for val in
sol])

# -----
# Step D: General Solution Format
# -----
print("\n--- General Solution (x = x_p + x_n) ---")
if not null_sols:
    print(f"x = {formatted_part_sol}")
else:
    general_sol_str = f"x =\n    {formatted_part_sol} <-- x_p"
    for i, sol in enumerate(null_sols):
        formatted_null_sol = ["{:.8f}".format(val) for val in sol]
        general_sol_str += f"\n    + c_{i+1} * {formatted_null_sol}
<-- v_{i+1}"

    print(general_sol_str)
    print("\n(Where c_1, c_2, etc., are arbitrary scalar constants
representing the free variables.)")

```

Pivot Columns (0-indexed): [0, 1, 2, 3]

```

Non-Pivot Columns (Free variables, 0-indexed): [4]
Number of non-pivot columns: 1
Number of free variables present: 1

--- Particular Solution (x_p) ---
['0.17470958', '1.67264309', '-1.12323560', '0.12519354',
'0.00000000']

--- Solutions to Ax = 0 (Null Space Basis Vectors) ---
Vector v_1: ['1.32482040', '-0.46520508', '-1.85869915', '-0.50460387',
'1.00000000']

--- General Solution (x = x_p + x_n) ---
x =
    ['0.17470958', '1.67264309', '-1.12323560', '0.12519354',
'0.00000000'] <-- x_p
    + c_1 * ['1.32482040', '-0.46520508', '-1.85869915', '-0.50460387',
'1.00000000'] <-- v_1

(Where c_1, c_2, etc., are arbitrary scalar constants representing the
free variables.)

```

1.3 Consider a random 5×7 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution.

```

# =====
# Verification Function
# =====

def multiply_matrix_vector(mat, vec):
    # Calculates A * x manually
    result = []
    for i in range(len(mat)):
        row_sum = 0.0
        for j in range(len(mat[0])):
            row_sum += mat[i][j] * vec[j]
        result.append([row_sum])
    return result

# =====
# PART 3: 5x7 Matrix
# =====

# 1. Setup specific 5x7 dimensions
m_part3 = 5
n_part3 = 7

random.seed(246) # Setting a random seed for reproducibility.

```

```

A_3 = [[get_random_float() for _ in range(n_part3)] for _ in
range(m_part3)]
b_3 = [[get_random_float()] for _ in range(m_part3)]

aug_3 = construct_augmented_matrix(A_3, b_3)

print("\n\n")
print("=== PART 3: 5x7 MATRIX ===")
print_matrix(A_3, "Random 5x7 Matrix A")
print_matrix(b_3, "Random 5x1 Vector b")

# 2. Compute REF and RREF
ref_3 = compute_REF(aug_3)
print_matrix(ref_3, "REF")

rref_3 = compute_RREF(ref_3)
print_matrix(rref_3, "RREF")

pivots_3, non_pivots_3, part_sol_3, null_sols_3 =
analyze_and_solve(rref_3)

if pivots_3 is not None:
    num_free_vars_3 = len(non_pivots_3)

    print(f"Pivot Columns (0-indexed): {pivots_3}")
    print(f"Non-Pivot Columns (Free variables, 0-indexed):
{non_pivots_3}")
    print(f"Number of free variables present: {num_free_vars_3}")

    print("\n--- Particular Solution (x_p) ---")
    formatted_part_sol_3 = [{"{: .8f}".format(v) for v in part_sol_3}]
    print(formatted_part_sol_3)

    print("\n--- Solutions to Ax = 0 (Null Space Basis Vectors) ---")
    if not null_sols_3:
        print("Only the trivial solution (all zeros) exists since
there are no free variables.")
    else:
        for i, sol in enumerate(null_sols_3):
            print(f"Vector v_{i+1}:", [{"{: .8f}".format(v) for v in
sol]})

# -----
# Step D: General Solution Format
# -----
print("\n--- General Solution Format (x = x_p + x_n) ---")
if not null_sols_3:
    print(f"x = {formatted_part_sol_3}")
else:
    general_sol_str = f"x =\n    {formatted_part_sol_3} <-- x_p"

```

```

        for i, sol in enumerate(null_sols_3):
            formatted_null_sol = ["{: .8f}".format(v) for v in sol]
            general_sol_str += f"\n + c_{i+1} * {formatted_null_sol}"
<-- v_{i+1}"

        print(general_sol_str)
        print("\n(Where c_1, c_2, etc., are arbitrary scalar constants
representing the free variables.)")

# =====
# 4. Constructing and Verifying the General Solution
# =====
print("\n=== GENERAL SOLUTION VERIFICATION ===")

# generating random scalars (c) to multiply with our null space
vectors.
random_scalars = [get_random_float() for _ in
range(len(null_sols_3))]

print(f"Random scalars chosen for null space vectors:
{['{: .8f}'.format(c) for c in random_scalars]}")

x_general = [0.0] * n_part3

# Adding the particular solution
for i in range(n_part3):
    x_general[i] = part_sol_3[i]
    # Adding the scaled null space vectors
    for j in range(len(null_sols_3)):
        x_general[i] += random_scalars[j] * null_sols_3[j][i]

print("\n--- Generated General Solution Vector (x_gen) ---")
print(["{: .8f}".format(v) for v in x_general])

# Verification: If x_general is valid, then A * x_general must
exactly equal vector b
b_verification = multiply_matrix_vector(A_3, x_general)

print("\n--- Verification: A * x_gen ---")
print_matrix(b_verification, "Result of A * x_gen (Should match
original Vector b)")

# Programmatic check with a small tolerance for floating point
arithmetic
is_valid = True
for i in range(m_part3):
    if abs(b_verification[i][0] - b_3[i][0]) > 1e-5:
        is_valid = False
        break

```

```

    if is_valid:
        print("SUCCESS: The general solution is VERIFIED. A * x_gen ==
b.")
    else:
        print("FAILED: The general solution did not produce vector
b.")

```

=== PART 3: 5x7 MATRIX ===

--- Random 5x7 Matrix A ---

```

['9.34367297', '5.65009134', '7.48549565', '1.59101978', '4.96588180',
'6.18018152', '4.00588679']
['6.03414293', '7.16624093', '8.50769760', '1.87849285', '7.19616802',
'4.09569366', '2.94095457']
['9.95570767', '7.10553352', '5.04140597', '1.84313517', '9.72386355',
'1.26426491', '2.87422558']
['1.89955866', '8.13257100', '8.82096748', '8.47639091', '2.47291916',
'5.54227325', '8.75706302']
['8.35676154', '5.43491878', '6.12777242', '2.16364130', '9.25208680',
'2.11299329', '2.32845453']

```

--- Random 5x1 Vector b ---

```

['4.27877312']
['2.93012920']
['1.28600825']
['4.79316718']
['1.63669697']

```

--- REF ---

```

['1.00000000', '0.60469703', '0.80112988', '0.17027777', '0.53146999',
'0.66142956', '0.42872720', '0.45793267']
['0.00000000', '1.00000000', '1.04439422', '0.24194275', '1.13412971',
'0.02971876', '0.10062889', '0.04744909']
['-0.00000000', '-0.00000000', '1.00000000', '0.02819419', '-
0.78707672', '1.31589732', '0.36954204', '0.81725279']
['0.00000000', '0.00000000', '0.00000000', '1.00000000', '-
0.99847041', '0.62995202', '1.11989256', '0.55509948']
['0.00000000', '0.00000000', '0.00000000', '0.00000000', '1.00000000',
'-0.60121193', '-0.39428454', '-0.41796519']

```

--- RREF ---

```

['1.00000000', '0.00000000', '0.00000000', '0.00000000', '0.00000000',
'0.40719025', '0.28304496', '0.27928258']
['0.00000000', '1.00000000', '0.00000000', '0.00000000', '0.00000000',
'-0.17483983', '0.33164266', '-0.01775967']
['0.00000000', '0.00000000', '1.00000000', '0.00000000', '0.00000000',
'0.84186118', '0.03873492', '0.48439771']

```

```

['0.00000000', '0.00000000', '0.00000000', '1.00000000', '0.00000000',
'0.02965970', '0.72621112', '0.13777361']
['0.00000000', '0.00000000', '0.00000000', '0.00000000', '1.00000000',
'-0.60121193', '-0.39428454', '-0.41796519']
Pivot Columns (0-indexed): [0, 1, 2, 3, 4]
Non-Pivot Columns (Free variables, 0-indexed): [5, 6]
Number of free variables present: 2

```

```

--- Particular Solution (x_p) ---
['0.27928258', '-0.01775967', '0.48439771', '0.13777361', '-
0.41796519', '0.00000000', '0.00000000']

```

```

--- Solutions to Ax = 0 (Null Space Basis Vectors) ---
Vector v_1: ['-0.40719025', '0.17483983', '-0.84186118', '-
0.02965970', '0.60121193', '1.00000000', '0.00000000']
Vector v_2: ['-0.28304496', '-0.33164266', '-0.03873492', '-
0.72621112', '0.39428454', '0.00000000', '1.00000000']

```

```

--- General Solution Format (x = x_p + x_n) ---
x =
    ['0.27928258', '-0.01775967', '0.48439771', '0.13777361', '-
0.41796519', '0.00000000', '0.00000000'] <-- x_p
    + c_1 * ['-0.40719025', '0.17483983', '-0.84186118', '-0.02965970',
'0.60121193', '1.00000000', '0.00000000'] <-- v_1
    + c_2 * ['-0.28304496', '-0.33164266', '-0.03873492', '-0.72621112',
'0.39428454', '0.00000000', '1.00000000'] <-- v_2

```

(Where c_1, c_2, etc., are arbitrary scalar constants representing the free variables.)

```

=== GENERAL SOLUTION VERIFICATION ===
Random scalars chosen for null space vectors: ['2.80260162',
'5.26064915']

```

```

--- Generated General Solution Vector (x_gen) ---
['-2.35090968', '-1.27240899', '-2.07877463', '-3.76569260',
'3.34118495', '2.80260162', '5.26064915']

```

```

--- Verification: A * x_gen ---

```

```

--- Result of A * x_gen (Should match original Vector b) ---
['4.27877312']
['2.93012920']
['1.28600825']
['4.79316718']
['1.63669697']

```

SUCCESS: The general solution is VERIFIED. A * x_gen == b.

Q2. Consider a dataset $X \in \mathbb{R}^{500 \times 6}$ constructed as follows: the first four features f_1, f_2, f_3, f_4 are generated as random features sampled from a standard normal distribution. The fifth and sixth features are defined by the relations $f_5 = 2f_1 + 3f_2$, $f_6 = f_3 - 2f_4$.

2.1 Write a Python code to generate the dataset $X = [f_1, f_2, f_3, f_4, f_5, f_6]$.

```
# =====  
# Q2.1 Generate Dataset X [500x6]  
# =====  
  
# Define number of data points  
n_samples = 500  
  
np.random.seed(246) # Setting a random seed for reproducibility.  
  
# 1. Generate the first four features from a standard normal  
# distribution  
# np.random.randn samples from a normal distribution with mean 0 and  
# variance 1  
f1 = np.random.randn(n_samples)  
f2 = np.random.randn(n_samples)  
f3 = np.random.randn(n_samples)  
f4 = np.random.randn(n_samples)  
  
# 2. Construct the 5th and 6th features based on the given relations  
f5 = 2 * f1 + 3 * f2  
f6 = f3 - 2 * f4  
  
# 3. Stack the 1D feature arrays as columns to form the 500x6 matrix X  
X = np.column_stack((f1, f2, f3, f4, f5, f6))  
  
# 4. Enforce the 8 decimal place constraint (n.dddddddd) across the  
# entire dataset  
X = np.round(X, 8)  
  
# Print the shape and the first 5 rows to verify construction and
```

```

formatting
print("\n\n")
print(f"Dataset X shape: {X.shape}")
print("\nFirst 10 rows of Dataset X:")
for row in X[:10]:
    # Using string formatting to ensure exactly 8 decimal places are
    # displayed
    print(["{:.8f}".format(val) for val in row])

```

Dataset X shape: (500, 6)

First 10 rows of Dataset X:

```

['0.87355171', '1.41238553', '1.23947064', '0.57195526', '5.98426001',
'0.09556012']
['-0.04310023', '-1.13976446', '-1.83174343', '-0.90485131', '-
3.50549384', '-0.02204081']
['0.79075838', '-0.38241763', '0.54674550', '0.15955685',
'0.43426386', '0.22763181']
['1.68463877', '-0.65154481', '0.55120589', '0.12468476',
'1.41464310', '0.30183637']
['-1.16749113', '0.75892477', '-0.50000164', '1.68654601', '-
0.05820795', '-3.87309367']
['2.40384057', '0.60340585', '-0.57628965', '0.21368891',
'6.61789871', '-1.00366747']
['-0.18161400', '0.35713113', '1.37748286', '1.80800544',
'0.70816539', '-2.23852803']
['-0.42559919', '1.04405512', '0.55165651', '-0.16548844',
'2.28096697', '0.88263339']
['0.50524632', '-0.98369448', '0.84011067', '0.15302575', '-
1.94059081', '0.53405916']
['-0.04187899', '-0.39800424', '0.75333508', '-2.12570146', '-
1.27777070', '5.00473801']

```

2.2 Write a Python code which computes the rank of X and display the output for the dataset X generated in step 1.

```

# =====
# Q2.2: Compute Rank of X via REF
# =====

# 1. Convert dataset X to a standard list of lists for your custom
# function
X_list = [row[:] for row in X.tolist()]

# 2. Compute the REF matrix using your existing function
ref_X = compute_REF(X_list, tol=1e-5)

```

```

# 3. Calculate rank by counting the non-zero rows (which equals the
number of pivots)
rank_X = 0
for row in ref_X:
    # Checking if the row has any non-zero elements (using a 1e-5
tolerance for ML dataset noise)
    if any(abs(val) > 1e-5 for val in row):
        rank_X += 1

print("\n\n")
print("=== Q2 PART 2: RANK OF DATASET X ===")
print(f"The computed rank of dataset X using our custom REF logic is:
{rank_X}")

```

```

=== Q2 PART 2: RANK OF DATASET X ===
The computed rank of dataset X using our custom REF logic is: 4

```

Observation: The rank is 4. Because f5 and f6 are constructed from f1-f4; Here, our REF logic verifies the same. 4 linearly independent pivot columns.

2.3 Numerical Experiment with the Power Method Read about the power method for finding the dominant eigenvalue and its corresponding eigenvector and perform the following tasks.

2.3a Write a Python code to compute the covariance matrix $C = (1/n)X^T X$, where n is number of data points

```

# =====
# Q2.3a: Compute Covariance Matrix C
# =====

# Convert the dataset X (numpy array) to a standard Python list for
manual operations
X_list = X.tolist()
n_samples = len(X_list)
n_features = len(X_list[0])

# 1. Transpose of X ( $X^T$ )
X_T = [[0.0 for _ in range(n_samples)] for _ in range(n_features)]
for i in range(n_samples):
    for j in range(n_features):
        X_T[j][i] = X_list[i][j]

# 2. Matrix Multiplication ( $X^T * X$ )
X_T_X = [[0.0 for _ in range(n_features)] for _ in range(n_features)]
for i in range(n_features):

```

```

    for j in range(n_features):
        row_col_sum = 0.0
        for k in range(n_samples):
            row_col_sum += X_T[i][k] * X_list[k][j]
        X_T_X[i][j] = row_col_sum

# 3. Calculating the final Covariance matrix C by dividing by n
C = [[0.0 for _ in range(n_features)] for _ in range(n_features)]
for i in range(n_features):
    for j in range(n_features):
        C[i][j] = X_T_X[i][j] / n_samples

print("\n\n")
print("=== Q2 PART 3(a): COVARIANCE MATRIX C ===")
print(f"Shape of Covariance Matrix C: ({n_features}, {n_features})")
print("\nCovariance Matrix C (6x6):")

for row in C:
    # Formatting the output to ensure 8 decimal places are
    # consistently displayed
    print(["{:.8f}".format(val) for val in row])

print("\n\n")

```

```

=== Q2 PART 3(a): COVARIANCE MATRIX C ===
Shape of Covariance Matrix C: (6, 6)

```

```

Covariance Matrix C (6x6):
['1.09431719', '0.09323680', '-0.07263640', '-0.03458211',
 '2.46834478', '-0.00347217']
['0.09323680', '0.92071436', '0.00040866', '-0.00145447',
 '2.94861668', '0.00331761']
['-0.07263640', '0.00040866', '0.92825024', '-0.03083406', '-
0.14404683', '0.98991837']
['-0.03458211', '-0.00145447', '-0.03083406', '0.96284940', '-
0.07352765', '-1.95653286']
['2.46834478', '2.94861668', '-0.14404683', '-0.07352765',
 '13.78253962', '0.00300848']
['-0.00347217', '0.00331761', '0.98991837', '-1.95653286',
 '0.00300848', '4.90298409']

```

2.3b Implement the Power Method in Python to approximate the largest eigenvalue λ_1 and its corresponding eigenvector v_1 of C . Show the code and the outputs.

```

# =====
# Q2.3b: Power Method (Largest Eigenvalue)
# =====

def power_method(A, tol=1e-7, max_iters=1000):
    n = len(A)
    np.random.seed(246) # Setting a random seed for reproducibility.
    v = np.random.rand(n)

    # normalisation
    v = v / np.sqrt(np.sum(v**2))

    lambda_old = 0.0
    iterations = 0

    for i in range(max_iters):
        w = np.dot(A, v)
        lambda_new = np.dot(v, w)

        # Calculate the magnitude of the new vector
        w_norm = np.sqrt(np.sum(w**2))

        # If the magnitude is practically zero, the matrix is
        depleted.
        if w_norm < 1e-10:
            # Return a zero vector so the subsequent deflation step
            # subtracts 0, leaving the matrix safely unharmed.
            v = np.zeros(n)
            break

        v = w / w_norm

        iterations += 1

        if abs(lambda_new - lambda_old) < tol:
            break

        lambda_old = lambda_new

    return lambda_new, v, iterations

# Execute the power method on our Covariance matrix C
lambda_1, v_1, iters_1 = power_method(C, tol=1e-7)

# Format the results to exactly 8 decimal places to satisfy
Instruction 6
lambda_1_rounded = np.round(lambda_1, 8)
v_1_rounded = np.round(v_1, 8)

```

```

print("\n\n")
print("=== Q2 PART 3(b): POWER METHOD (LARGEST EIGENVALUE) ===")
# Formatting strings to ensure trailing zeros are shown if necessary
print(f"Dominant Eigenvalue (lambda_1): {lambda_1_rounded:.8f}")
print("Corresponding Eigenvector (v_1):")
print(["{:.8f}".format(val) for val in v_1_rounded])
print(f"\nThe algorithm converged in {iters_1} iterations.")

print("\n\n")

```

```

=== Q2 PART 3(b): POWER METHOD (LARGEST EIGENVALUE) ===
Dominant Eigenvalue (lambda_1): 14.85835172
Corresponding Eigenvector (v_1):
['0.17417140', '0.20491301', '-0.01082328', '-0.00557749',
'0.96308183', '0.00033171']

```

The algorithm converged in 12 iterations.

2.3c Write a Python code to obtain the next largest eigenvalue λ_2 and its corresponding eigenvector v_2 by applying power method on $C - v_1 v_1^T C$. Having found out $v_1, v_2, \dots, v_{k-1}$, one can find λ_k and its corresponding eigenvector v_k by applying power method on $C - \sum_{j=1}^{k-1} v_j v_j^T C$ ($j=1$ to $k-1$). Give the code and also display the obtained eigenvalues and the corresponding eigenvectors.

```

# =====
# Q2.3c: Deflation: Subsequent eigenvalues/eigenvectors
# =====

# Storing all the eigenvalues and eigenvectors in lists.
# Initializing with the dominant ones we found in Part 3(b).
eigenvalues = [lambda_1]
eigenvectors = [v_1]

# C_deflated will keep track of our matrix as we strip away the
# dominant components
C_deflated = np.copy(C)

# Since C is a 6x6 matrix, it has 6 eigenvalues in total
num_eigenvalues = len(C)

print("\n\n")

print("=== Q2 PART 3(c): ALL EIGENVALUES VIA DEFLATION ===")
print(f"Eigenvalue 1: {np.round(lambda_1, 8):.8f}")

```

```

print("Eigenvector 1:", [{":.8f}".format(val) for val in np.round(v_1,
8)])
print("-" * 50)

for k in range(1, num_eigenvalues):
    # Retrieve the most recently found eigenvector
    v_prev = eigenvectors[-1]

    # Calculate the deflation term: v_prev * v_prev^T * C
    # np.outer creates the matrix resulting from v * v^T
    deflation_term = np.dot(np.outer(v_prev, v_prev), C)

    # Subtract this term from our current deflated matrix.
    # This matches the assignment formula: C_new = C_old - v_prev *
    v_prev^T * C.
    C_deflated = C_deflated - deflation_term

    # Apply the power method on the newly deflated matrix
    l_k, v_k, i_k = power_method(C_deflated, tol=1e-12)

    # Store the results
    eigenvalues.append(l_k)
    eigenvectors.append(v_k)

    # Format to 8 decimal places for printing[cite: 1]
    l_k_rounded = np.round(l_k, 8)
    v_k_rounded = np.round(v_k, 8)

    #print("\n\n")
    print(f"Eigenvalue {k+1}: {l_k_rounded:.8f}")
    print(f"Eigenvector {k+1}:", [{":.8f}".format(val) for val in
v_k_rounded])
    print("-" * 50)

#print("\n")

```

=== Q2 PART 3(c): ALL EIGENVALUES VIA DEFLATION ===

Eigenvalue 1: 14.85835172

Eigenvector 1: ['0.17417140', '0.20491301', '-0.01082328', '-0.00557749', '0.96308183', '0.00033171']

Eigenvalue 2: 5.88361216

Eigenvector 2: ['-0.00096657', '0.00054781', '0.18464893', '-0.36410947', '-0.00028972', '0.91286787']

```

Eigenvalue 3: 0.98848540
Eigenvector 3: ['-0.65467675', '0.44679520', '0.54423322',
'0.27315796', '0.03103209', '-0.00208271']
-----
Eigenvalue 4: 0.86120560
Eigenvector 4: ['0.50531916', '-0.33989302', '0.70917640',
'0.35512145', '-0.00904076', '-0.00106650']
-----
Eigenvalue 5: -0.00000000
Eigenvector 5: ['0.00000000', '0.00000000', '0.00000000',
'0.00000000', '0.00000000', '0.00000000']
-----
Eigenvalue 6: -0.00000000
Eigenvector 6: ['0.00000000', '0.00000000', '0.00000000',
'0.00000000', '0.00000000', '0.00000000']
-----

```

Observation: Here, the Eigenvalues 5 and 6 are 0. This perfectly aligns with the finding in Part 2 that the rank of the matrix is 4. Rank of 4 means there are exactly 4 non-zero eigenvalues.

2.3d Find all the eigenvalues and eigenvectors using Python function and compare with the obtained result in (c).

```

# =====
# Q2.3d: Verification with Built-in Eigenvalues Function
# =====

print("\n\n")
print("== Q2 PART 3(d): EXACT EIGENVALUES (BUILT-IN) ==")

# Using np.linalg.eigh because our covariance matrix C is symmetric
eigvals_exact, eigvecs_exact = np.linalg.eigh(C)

# eigh returns values in ascending order. We reverse them to match
# the descending order (largest to smallest) of our Power Method
# results.
eigvals_exact = eigvals_exact[::-1]
eigvecs_exact = eigvecs_exact[:, ::-1]

for i in range(len(eigvals_exact)):
    exact_val = np.round(eigvals_exact[i], 8)
    approx_val = np.round(eigenvalues[i], 8)

    print(f"\nEigenvalue {i+1}:")
    print(f"    Exact (eigh function) : {exact_val:.8f}")
    print(f"    Part 3c : {approx_val:.8f}")

# Using abs() to compare the eigenvectors because the power method

```

```

might
    # find the exact opposite direction (multiplied by -1), which is
    # still a valid eigenvector.
    diff = abs(approx_val - exact_val)
    print(f"    Absolute Difference : {diff:.8f}")

```

=== Q2 PART 3(d): EXACT EIGENVALUES (BUILT-IN) ===

```

Eigenvalue 1:
    Exact (eigh function) : 14.85835173
                        Part 3c : 14.85835172
    Absolute Difference : 0.00000001

Eigenvalue 2:
    Exact (eigh function) : 5.88361216
                        Part 3c : 5.88361216
    Absolute Difference : 0.00000000

Eigenvalue 3:
    Exact (eigh function) : 0.98848540
                        Part 3c : 0.98848540
    Absolute Difference : 0.00000000

Eigenvalue 4:
    Exact (eigh function) : 0.86120560
                        Part 3c : 0.86120560
    Absolute Difference : 0.00000000

Eigenvalue 5:
    Exact (eigh function) : -0.00000000
                        Part 3c : -0.00000000
    Absolute Difference : 0.00000000

Eigenvalue 6:
    Exact (eigh function) : -0.00000000
                        Part 3c : -0.00000000
    Absolute Difference : 0.00000000

```

Observation: The eigenvalues obtained from the deflation power method match the built-in numpy function `np.linalg.eigh()`. As expected, the 5th and 6th eigenvalues are effectively zero in both methods, confirming our rank of 4 from Part 2.

2.3e Compare the number of iterations required to get an accuracy of 10^{-7} using the power method. The actual values can be taken as the one obtained in (d)

```

# =====
# Q2.3e: Compare Iterations for  $10^{-7}$  accuracy

```

```

# =====

print("\n\n")
print("\n=== Q2 PART 3(e): ITERATIONS FOR 10^-7 ACCURACY ===")

def count_iterations_to_true_accuracy(A, exact_lambda, tol=1e-7,
max_iters=2000):
    n = A.shape[0]
    np.random.seed(246) # Setting a random seed for reproducibility.
    v = np.random.rand(n)

    v = v / np.sqrt(np.sum(v**2))

    for i in range(1, max_iters + 1):
        w = np.dot(A, v)

        # Guard against zero vectors using custom norm
        if np.sqrt(np.sum(w**2)) < 1e-15:
            return i, 0.0

        lambda_new = np.dot(v, w)

        v = w / np.sqrt(np.sum(w**2))

        if abs(lambda_new - exact_lambda) < tol:
            return i, lambda_new

    return max_iters, lambda_new

# Initialize a fresh copy of C for deflation
C_compare = np.copy(C)

# We will test the first 4 eigenvalues, as they are the non-zero valid ones (Rank = 4)
for k in range(4):
    target_lambda = eigvals_exact[k]

    # Get iterations to hit 10^-7 accuracy against the target
    iters, achieved_val = count_iterations_to_true_accuracy(C_compare,
target_lambda, tol=1e-7)

    print(f"Eigenvalue {k+1} (Exact target: {target_lambda:.8f})")
    print(f"  Iterations required: {iters}")

    # Deflate the matrix for the next eigenvalue loop using the vectors we found in 3(c)[cite: 1]
    v_prev = eigenvectors[k]
    deflation_term = np.dot(np.outer(v_prev, v_prev), C)
    C_compare = C_compare - deflation_term

```

=== Q2 PART 3(e): ITERATIONS FOR 10^{-7} ACCURACY ===

Eigenvalue 1 (Exact target: 14.85835173)

Iterations required: 11

Eigenvalue 2 (Exact target: 5.88361216)

Iterations required: 7

Eigenvalue 3 (Exact target: 0.98848540)

Iterations required: 52

Eigenvalue 4 (Exact target: 0.86120560)

Iterations required: 2

Comments: The number of iterations required fluctuates significantly between the eigenvalues. The Power Method's convergence speed relies on the ratio between the current eigenvalue and the next largest eigenvalue ($|\lambda_{k+1}/\lambda_k|$). If the next eigenvalue is very close in magnitude to the current one, it takes many more iterations to separate them and converge. If the gap is large, convergence is rapid.